CUSTOMER FEEDBACK & FEATURE REQUEST PORTAL

Project Report Submitted to the SRM University-AP Andhra Pradesh,
for partial fulfillment of the course

Bachelor of Technology

Computer Science and Engineering School of Engineering and Science

Submitted by:

1. K. Akhil _AP24110010222
2. K. Venkat_AP24110010156
3. P. Ram Charan_AP24110010159
4. U. Anshul_AP24110010173
5. V. Niteesh_AP24110010165

Department of Computer Science and Engineering SRM University-AP

Course :

Full Stack Development

Academic Year

2025 – 2026

INTRODUCTION:

The Customer Feedback & Feature Request Portal is a comprehensive web-based application developed to streamline the process of collecting, organizing, and analyzing customer feedback. In modern digital environments, businesses receive feedback through multiple channels such as emails, social media platforms, and customer support systems, which often leads to unstructured and scattered data. This makes it difficult to identify user needs and prioritize improvements effectively. The proposed system addresses this challenge by providing a centralized platform where users can submit feedback, suggest new feature ideas, and interact with existing requests through a voting mechanism. By enabling community participation and structured data storage, the system helps organizations make informed decisions and improve their products based on real user requirements. Additionally, the project demonstrates the implementation of full-stack development using modern technologies and follows best practices in software architecture.

SCENARIO BASED CASE STUDY:

Consider a user named Rahul, a regular customer of a popular project management application used for organizing tasks and team collaboration. Rahul frequently uses the application during late hours and finds the bright interface uncomfortable for his eyes. He realizes that the application does not provide a dark mode feature, which is increasingly common in modern applications.

To address this issue, Rahul visits the Customer Feedback & Feature Request Portal. After registering and logging into the system, he navigates to the feature request section and submits a new request titled **"Add Dark Mode Feature"**. In the description, he explains how a dark mode would reduce eye strain and improve user experience, especially during nighttime usage.

Once the request is submitted, it becomes visible to other users on the platform. Many users who face the same issue come across Rahul's request while browsing the portal. They read the description and find it relevant to their needs, so they start upvoting the request. Within a short period, the feature request gains significant attention and receives over 200 votes, making it one of the most popular suggestions on the platform.

On the administrative side, the admin regularly monitors the dashboard to review feedback and feature requests. The system highlights trending requests based on the number of votes. The admin notices that the "Add Dark Mode Feature" request has received a high number of votes and positive engagement from users.

After analyzing its popularity and importance, the admin marks the request as **"Under Review"** and forwards it to the development team for further consideration. Eventually, the feature is

approved and implemented in the application. Once completed, the admin updates the status to **"Implemented"**, and users are notified about the new feature.

As a result, Rahul and many other users benefit from the newly added dark mode, leading to improved user satisfaction. This scenario demonstrates how the portal effectively collects user feedback, prioritizes feature development based on community interest, and enhances the overall product experience.

SYSTEM REQUIREMENTS:

To ensure smooth development, deployment, and execution of the **Customer Feedback & Feature Request Portal**, certain system requirements must be fulfilled. These requirements are categorized into software and hardware specifications.

1. Software Requirements :

These are the essential tools, technologies, and platforms required for developing and running the application efficiently:

- **Operating System:**
  Windows 10/11, macOS, or Linux – Supports cross-platform development and testing.
- **Node.js (v16 or above):**
  Provides the runtime environment for executing JavaScript on the server side and handling backend operations.
- **npm (Node Package Manager):**
  Used to install and manage project dependencies such as React libraries and backend packages.
- **React.js:**
  A JavaScript library used to build dynamic and responsive user interfaces for the frontend.
- **Express.js:**
  A lightweight framework used to develop RESTful APIs and handle server-side logic.
- **MongoDB:**
  A NoSQL database used to store application data such as users, feedback, feature requests, and votes.
- **Mongoose:**
  An Object Data Modeling (ODM) library used to define schemas and interact with MongoDB.

- **Browser:**
  Google Chrome / Mozilla Firefox (latest versions) for testing and running the frontend interface.
- **Postman / Thunder Client:**
  Tools used for testing API endpoints during development.
- **Visual Studio Code:**
  A powerful code editor with support for extensions, debugging, and integrated terminal.
- **Git & GitHub:**
  Version control system and repository hosting platform for managing source code.

2. Hardware Requirements :

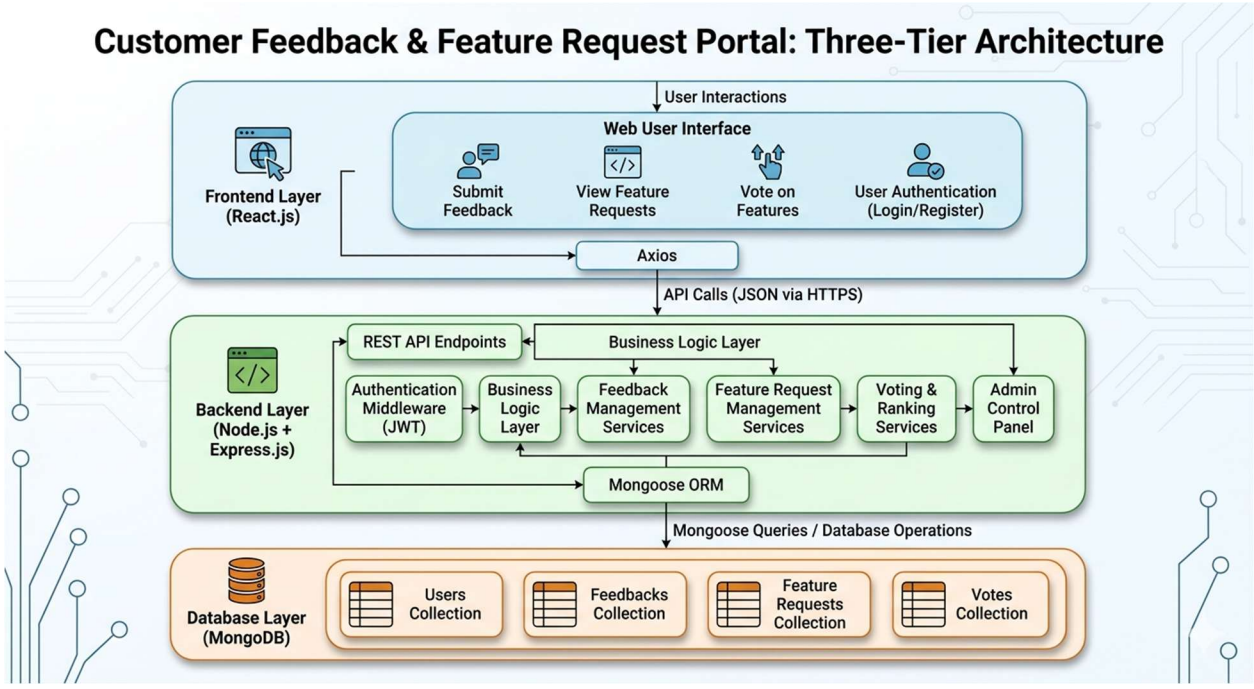
These specifications ensure that the system runs efficiently during development and usage:

- **Processor:**
  Intel Core i5 (8th Generation or above) or AMD Ryzen 5 – Required for smooth multitasking and development.
- **RAM:**
  Minimum 8 GB (16 GB recommended) – Helps in running development servers, databases, and browsers simultaneously.
- **Storage:**
  At least 1 GB of free disk space for project files, dependencies, and database setup.
- **Display:**
  Resolution of 1366 × 768 or higher for better development and user experience.
- **Internet Connection:**
  Required for downloading dependencies, accessing APIs, and using cloud services if integrated.

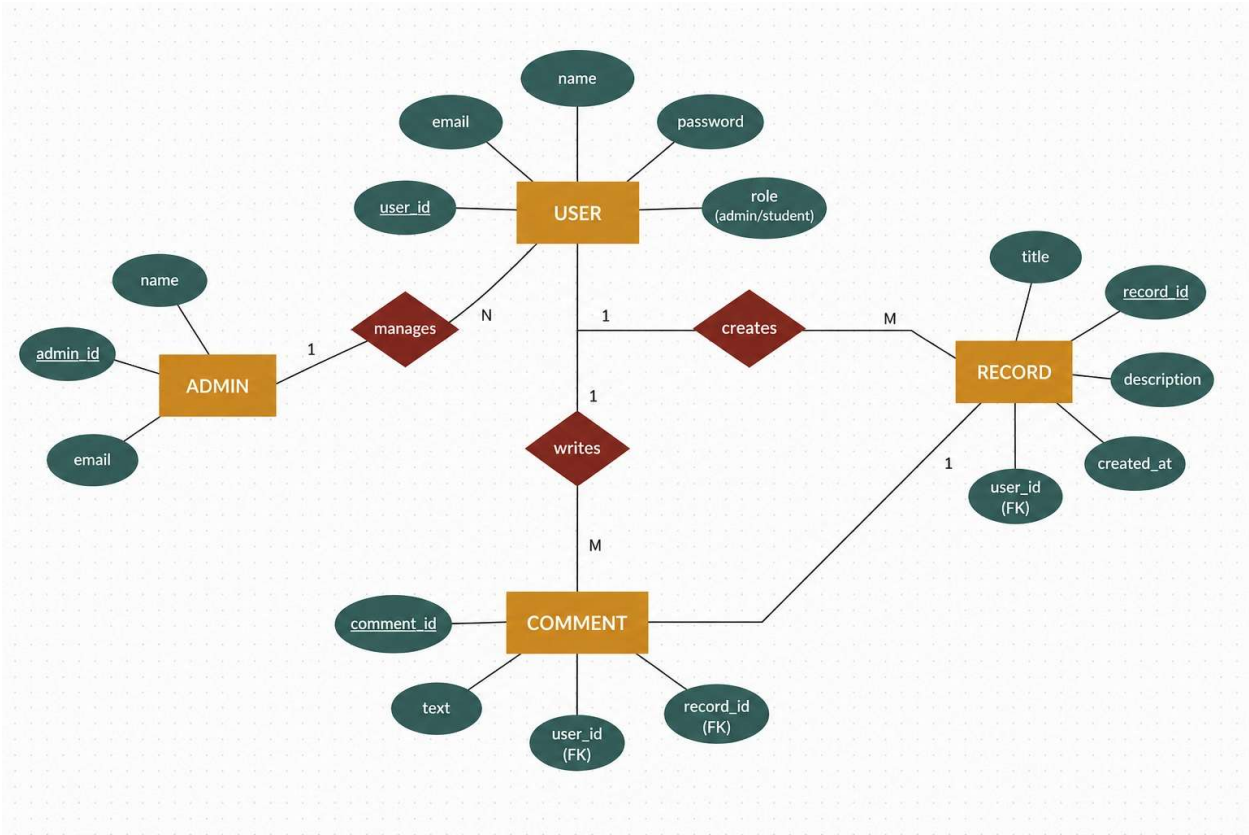
PROJECT ARCHITECTURE:

TECHNICAL ARCHITECTURE:

The proposed Customer Feedback & Feature Request Portal will follow a three-tiered architectural approach consisting of frontend, backend, and database layers to ensure scalability, maintainability, and optimized performance. In particular, the frontend layer of the portal, developed with React.js technology, will provide an interface through which users interact with the system via activities such as submitting customer feedbacks, viewing requested features, and voting. The frontend will communicate with the backend through the exchange of API calls using Axios. The backend will be the main component of the Customer Feedback & Feature Request Portal, developed with Node.js and Express.js, responsible for the implementation of all business logic related to the management of the application's operation, processing client requests, performing authentication using JWT, and implementing functions such as submitting feedback, voting for features, and controlling administration. Finally, the database layer, designed with MongoDB, will be utilized to store all application data, including the information of registered users, submitted feedbacks, feature requests, and voting records. The interaction between different architectural components can be described in the following order. First, the user will interact with the frontend, creating a request that will be sent to the backend.



ER DIAGRAM :



ER Diagram of Customer Feedback & Feature Request Portal is a visual representation that defines the database schema and the relationship between entities in the system. Some of the major entities included in the Customer Feedback & Feature Request Portal ER Diagram are User, Feedback, FeatureRequest, and Vote. User entity holds the record of the registered users in the system and its attributes include user_id, user_name, user_email, password, role, and creation_date. There can be a one-to-many relationship between User entity and Feedback entity as the user creates more than one feedback in the portal, and similarly, the user creates more than one FeatureRequest hence making a one-to-many relationship between User and FeatureRequest. The attributes of Feedback entity include feedback_id, feedback_title, description, category, and user_id of the user that creates the Feedback whereas FeatureRequest entity attributes are request_id, title, description, no_of_votes, request_status (pending, under_review, implemented), and user_id of the creator of the request. Another entity Vote is used to provide a many-to-one relationship with User and FeatureRequest entity.

Key Features:

There are some essential features that ensure efficiency and friendliness of the Customer Feedback & Feature Request Portal. First, there is safe user authentication that guarantees safety due to encryption, usage of JWT-based authorization, and role-based access both for users and admin. Users can provide their feedback on any product or service through provision of such information as title, description, and category stored in a database. Users may not only give their feedback but also post feature requests that are later reviewed. Moreover, one of the important features of the customer feedback portal is the possibility for upvoting feature requests submitted by users. Admins are provided with the dashboard that shows all activity performed in the portal. They can view feedback, control submission from users, and change the status of certain requests, whether they are pending, under review, or already implemented. There is one more important aspect – community visibility, since all feedback is shown publicly.

ROLES AND RESPONSIBILITIES

User:

● **Registration and Account Management:**
Users are responsible for creating an account on the platform by providing accurate details such as username, email, and password. They must manage their login credentials securely and update their profile information when required.

● **Feedback Submission:**
Users can submit feedback about products or services by providing details such as title, description, and category. They are responsible for submitting clear and meaningful feedback.

● **Feature Request Submission:**
Users can suggest new features by creating feature requests. They should provide proper descriptions to explain their ideas clearly.

● **Feature Voting:**
Users can browse existing feature requests and upvote the ones they support. This helps in identifying popular features based on community interest.

● **Feedback Discovery:**
Users can view feedback and feature requests submitted by other users. They can explore trending and popular suggestions.

● **Communication and Interaction:**
Users may interact with the platform by participating in discussions (if enabled) and following updates on their submitted feedback or feature requests.

Admin:

● System Management:
The admin is responsible for managing the overall system, ensuring proper functioning of the platform, maintaining data security, and handling application configurations.

● User Management:
Admins manage user accounts, including monitoring registrations, handling user roles (user/admin), and taking necessary actions like restricting or managing users if required.

● Feedback and Feature Request Management:
Admins review feedback and feature requests submitted by users. They ensure that the content is appropriate and relevant.

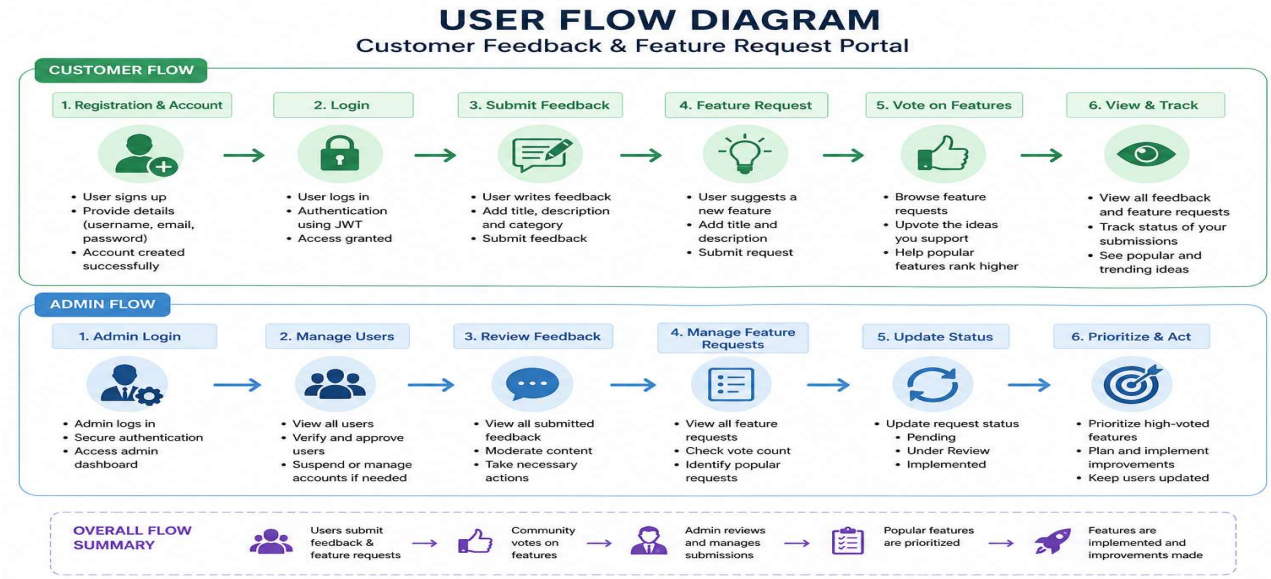
● Moderation of Submissions:
Admins are responsible for moderating user submissions, removing inappropriate content, and maintaining quality and consistency of data.

● Feature Prioritization:
Admins analyze feature requests based on votes and user interest. They identify popular requests and prioritize them for implementation.

● Status Management:
Admins update the status of feature requests such as:

- Pending
- Under Review
- Implemented

User Flow:



Customer Flow

● **Registration & Account** – Users create an account by providing details such as username, email, and password, and manage their account securely.

● **Feedback Submission** – Users can submit feedback about products or services by providing title, description, and category.

● **Feature Request** – Users can suggest new features by creating feature requests with proper descriptions.

● **Voting** – Users can browse feature requests and upvote ideas they support to highlight popular features.

● **Feedback Discovery** – Users can view feedback and feature requests submitted by other users and explore trending ideas.

Admin Flow

● **System Management** – Admin monitors system performance, ensures data security, and manages overall application settings.

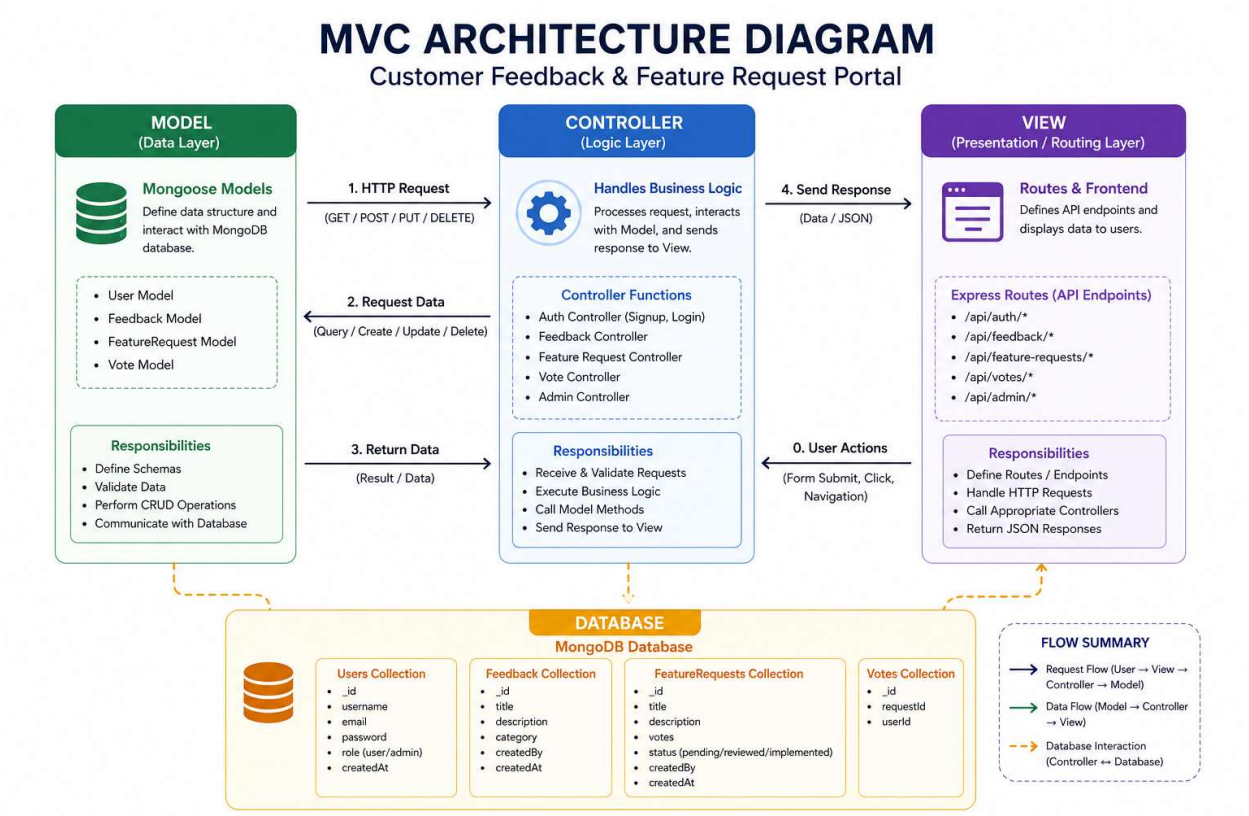
● **User Management** – Admin manages user accounts, verifies users, and handles any misuse or moderation.

● **Feedback Management** – Admin reviews and moderates feedback submitted by users.

● **Feature Request Management** – Admin analyzes feature requests, monitors votes, and identifies popular features.

● **Status Management** – Admin updates the status of feature requests such as Pending, Under Review, or Implemented.

MVC ARCHITECTURE:



The Customer Feedback & Feature Request Portal follows the Model-View-Controller (MVC) architectural pattern. This design separates the application into three layers, making the system modular, scalable, and easy to maintain.

Model Layer (Data Layer)

The Model layer handles all data-related operations. It defines schemas and interacts with the MongoDB database using Mongoose.

In this project, models include:

- User (stores user details and roles)
- Feedback (stores user feedback)
- FeatureRequest (stores feature suggestions and votes)
- Vote (stores voting data)

This layer ensures structured data storage and validation.

Controller Layer

The Controller layer contains the business logic of the application. It processes incoming requests, interacts with models, and sends responses back to the client.

In this project, controllers handle:

- User authentication (signup, login using JWT)
- Feedback submission and retrieval
- Feature request creation and voting
- Admin actions like reviewing and updating status

View Layer (Routing Layer)

In this backend REST API, the View is implemented as the routing layer. It defines API endpoints and maps them to controller functions.

Examples:

- POST /api/users/register
- POST /api/feedback
- GET /api/features
- PUT /api/features/:id/status

The routing layer handles HTTP methods such as GET, POST, PUT, and DELETE.

Advantages of Using MVC in This Project

● **Separation of Concerns** – Each layer has a specific role, improving clarity and maintainability.

● **Scalability** – New features like comments or analytics can be added easily.

● **Reusability** – Controllers and models can be reused across different parts of the application.

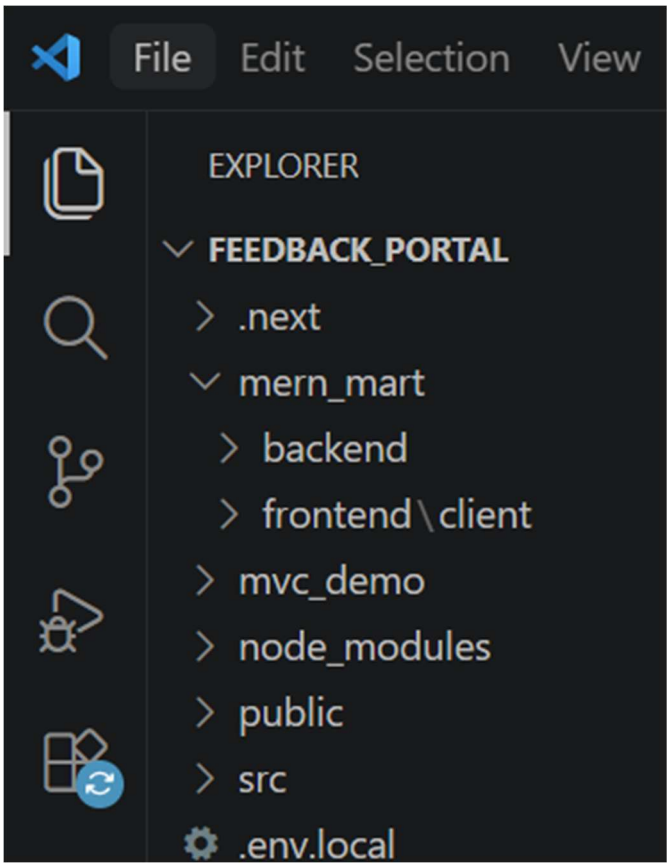
● **Testing** – Each layer can be tested independently.

● **Collaboration-Friendly** – Developers can work on frontend, backend, and database separately.

Project Setup And Configuration

- Creating Project Folder

1. Create a new folder with your project name.
2. Inside that folder create two folders:
3. Name one as **Client**
4. Name another one as **Server**
5. Open the folder in VS Code



- Client Setup (React Application)

  - Open Client folder in terminal

    - Run:

```
npm create vite@latest . -- --template react
```

  - Select:

- Framework → React

- Variant → JavaScript

○ Navigate to client folder:

```
cd client
```

○ Install dependencies:

```
npm install
```

○ Run frontend:

```
npm run dev
```

● Server Setup (Node.js Backend)

○ Open Server folder in terminal

■ Initialize project:

```
npm init -y
```

○ Install dependencies:

```
npm install express mongoose cors dotenv jsonwebtoken bcryptjs
```

○ Create files:

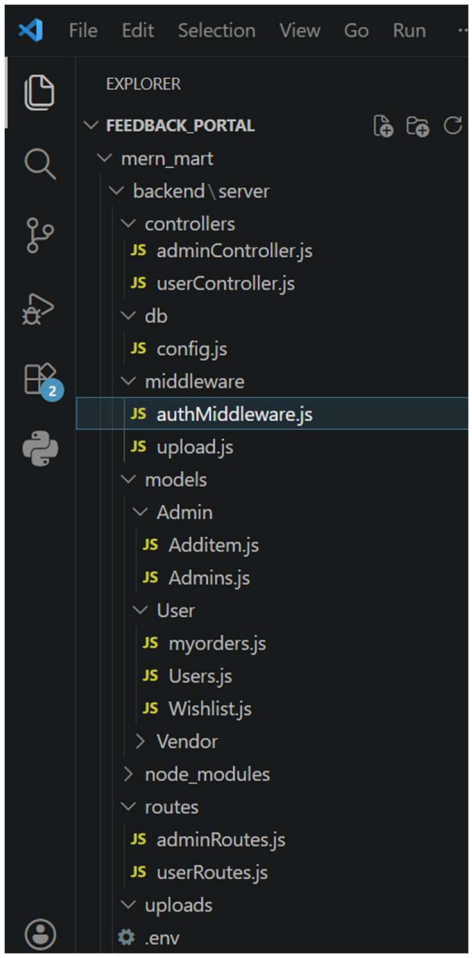
- server.js

○ Create folders:

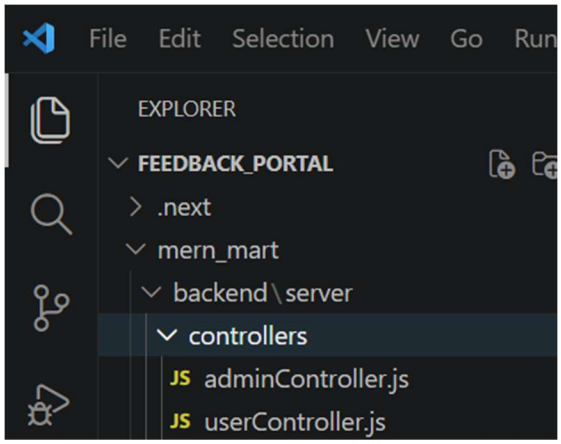
- models
- controllers
- routes
- middleware

BACKEND DEVELOPMENT:BACKEND VIDEO REFERENCE:

https://drive.google.com/file/d/16v_UckxJhA78cNLw0w59S443mI0ykii2/view?usp=share_link



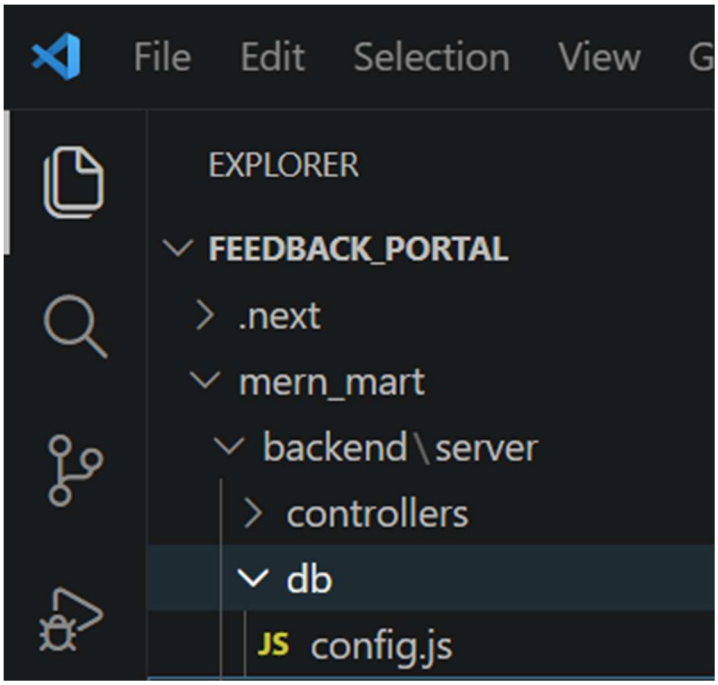
CONTROLLERS



● **adminController.js** → Handles all admin-related business logic (review feedback, manage feature requests, update status, moderate users).

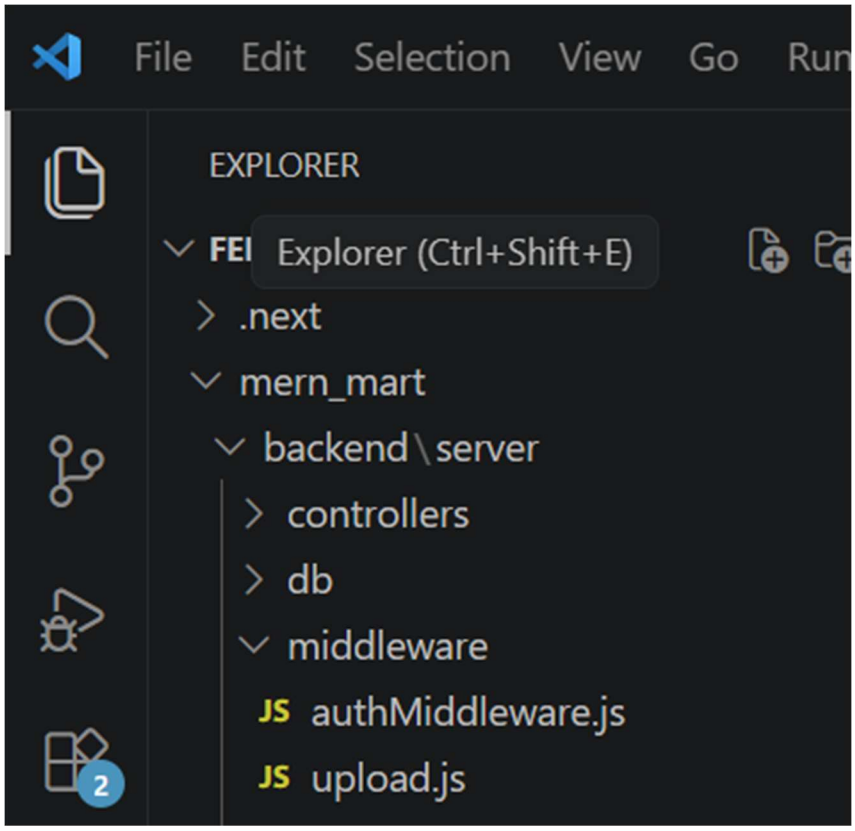
● **userController.js** → Handles all user-related operations (register, login, submit feedback, create feature requests, vote on features, view data).

DB



● **config.js** → Contains database connection setup using MongoDB (Mongoose configuration).

MIDDLEWARE



● **authMiddleware.js** → Middleware for authentication and authorization using JWT (verifies user/admin access).

● **upload.js** → Middleware for handling file uploads (if used for attachments or profile images).

MODELS

● **User**
  ○ **Users.js** → Schema/model for storing user details (username, email, password, role).

● **Feedback**
  ○ **Feedback.js** → Schema/model for storing user feedback (title, description, category, createdBy).

● **FeatureRequests**
  ○ **FeatureRequest.js** → Schema/model for storing feature requests (title, description, votes, status, createdBy).

● **Votes**
  ○ **Vote.js** → Schema/model for storing votes on feature requests (requestId, userId).

routes

● **adminRoutes.js** → Defines API routes for admin actions (review feedback, manage feature requests, update status, moderate users).

● **userRoutes.js** → Defines API routes for user actions (register, login, submit feedback, create feature requests, vote, view feedback).

DATABASE DEVELOPMENT:

mern_mart > backend > server > db > JS config.js > ...

```
const mongoose = require("mongoose");

const connectDB = async () => {
  try {
    const conn = await mongoose.connect(process.env.MONGO_URI, {
      useNewUrlParser: true,
      useUnifiedTopology: true,
    });
    console.log(`MongoDB Connected: ${conn.connection.host}`);
  } catch (error) {
    console.error(`Database connection error: ${error.message}`);
    process.exit(1);
  }
};

module.exports = connectDB;
```

Configure MongoDB:

● **Install Mongoose:**
In your Node.js project directory, run:

```
npm install mongoose
```

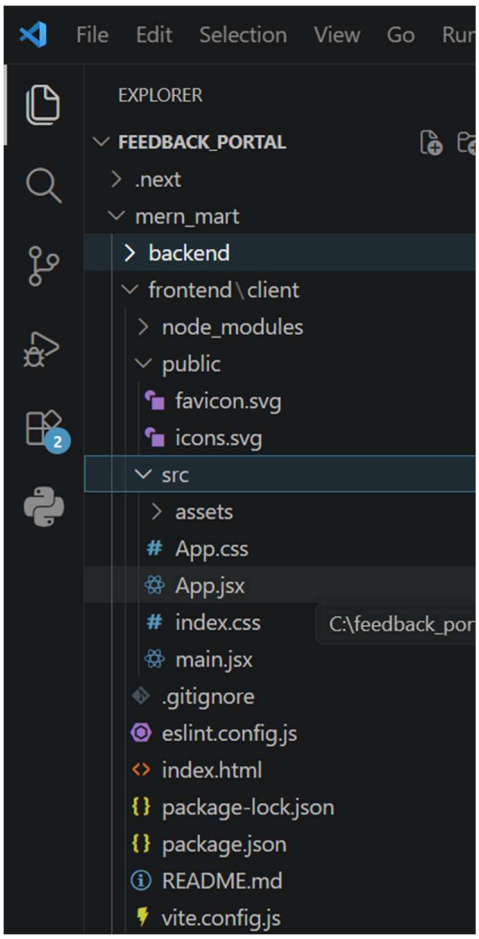
● **Create Database Connection:**
Create a connection file (config.js) inside the **db** folder to connect MongoDB with your backend using Mongoose.

- **Create Schemas & Models:**
Define schemas for collections like Users, Feedback, FeatureRequests, and Votes using Mongoose, and create models to interact with the database.

FRONTEND DEVELOPMENT :

FRONTEND VIDEO REFERENCE:

https://drive.google.com/file/d/1UVfjqvs_NpUuD5HECIfH-LBO65jOTNmw/view?usp=share_link



Main Files

- **App.jsx** → Main React component that handles routing and renders all pages/components.
- **main.jsx** → Entry point of the React application; renders App component to the DOM.
- **App.css** → Styling for main application layout and components.

- **index.css** → Global styles applied across the entire application

Assets

- **assets/** → Contains static files like images, icons, and other resources used in the UI.

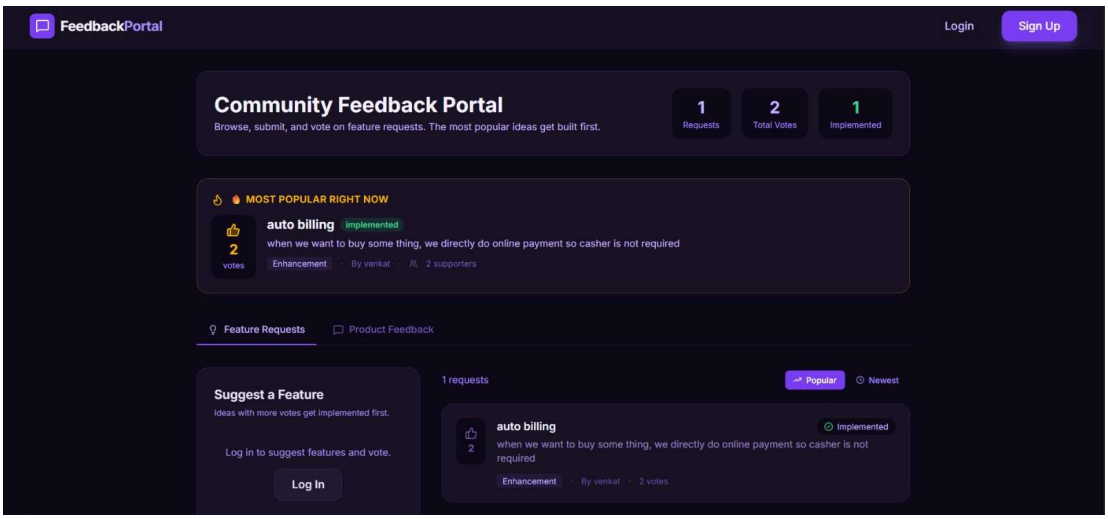
Public Folder

- **favicon.svg** → Application icon displayed in browser tab.
- **icons.svg** → Contains SVG icons used in the application.

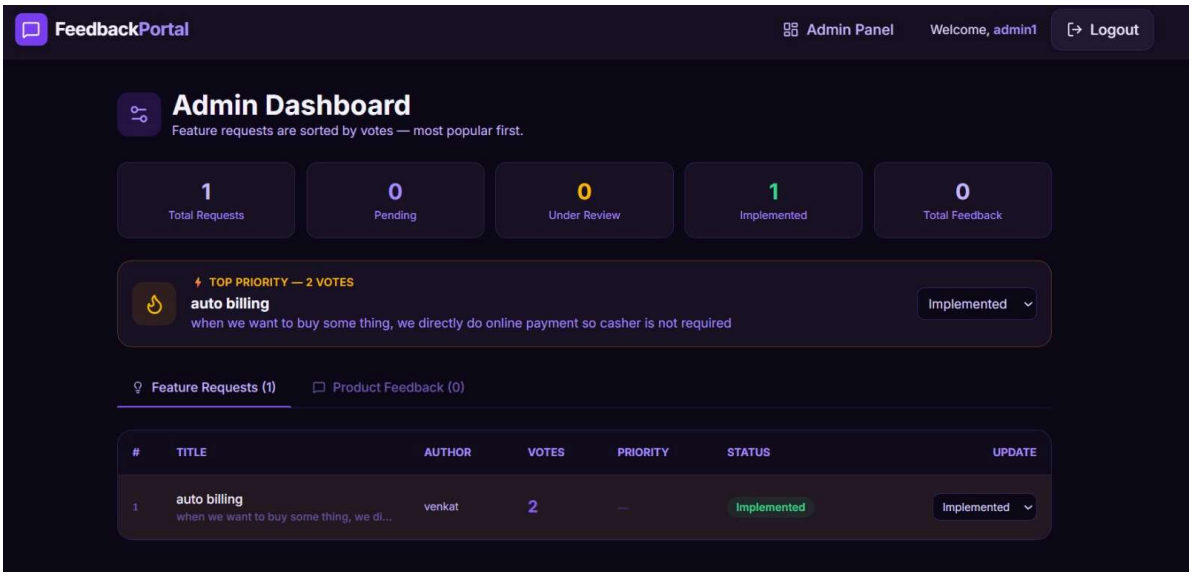
Configuration Files

- **index.html** → Main HTML file where React app is mounted.
- **package.json** → Contains project dependencies, scripts, and configurations.
- **package-lock.json** → Locks dependency versions for consistent installs.
- **eslint.config.js** → Configuration file for code linting and formatting rules.
- **.gitignore** → Specifies files and folders to be ignored by Git.

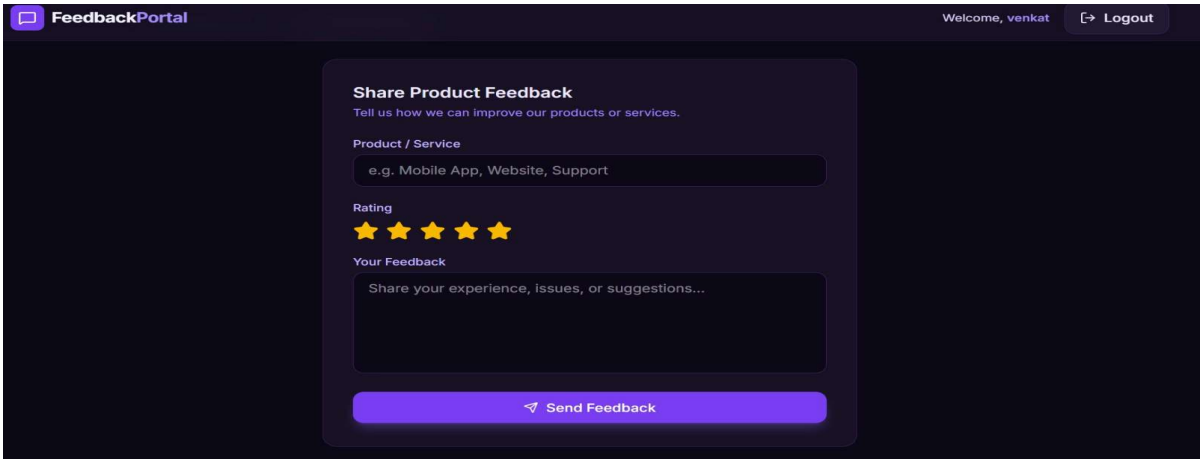
Landing page :



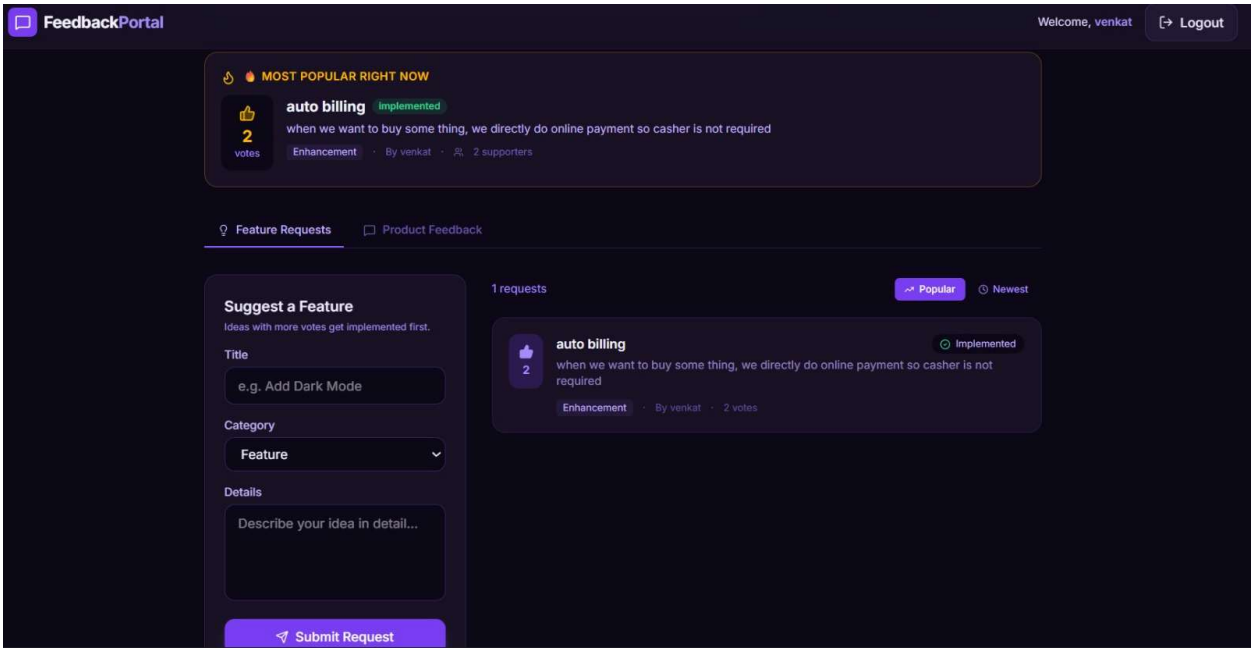
Admin DashBoard :



User page product feedback:



User page featurerequest:



DEMO VEDIO:

https://drive.google.com/file/d/1gg1WB7GSwpLmDVENIqTDLxcUbwoUm1VX/view?usp=share_link